

Laboratoire DACSC
(TCP/IP & HTTP en C/C++/Java/Vue.js)

Contexte général :

L'application à réaliser est une **application de gestion des consultations d'un hôpital**. Elle concerne les **patients** et les **médecins** qui en sont les principaux utilisateurs.

Les **patients** présents à l'hôpital pourront prendre des rendez-vous avec le médecin de leur choix à l'aide d'une borne interactive. Ces bornes disposeront d'une application (de type « **client lourd** ») dédiée à cet effet.

Les **médecins** de l'hôpital disposeront, dans leur cabinet d'examen, de deux applications. La première leur permettra de planifier des nouvelles plages horaires de consultation mais également d'attribuer à ces plages horaires des patients qu'ils auraient en face d'eux physiquement. La seconde application permettra aux médecins d'accéder au dossier médical d'un patient, c'est-à-dire à l'ensemble des rapports médicaux rédigés par les médecins qui ont eu affaire à ce patient. Cette seconde application permettra également au médecin de rédiger un nouveau rapport médical à la suite d'une consultation qu'il aurait eue avec un patient. Ces deux applications sont de type « **client lourd** » (desktop) car installée sur l'ordinateur du médecin.



En outre, un **patient** pourra accéder à toutes ses consultations à venir à l'aide d'une application web (de type « **client léger** ») en utilisant un browser internet sur son PC ou sa tablette. Grâce à cette application, il pourra également annuler un rendez-vous ou prendre rendez-vous avec un médecin pour une nouvelle consultation.

Consignes :

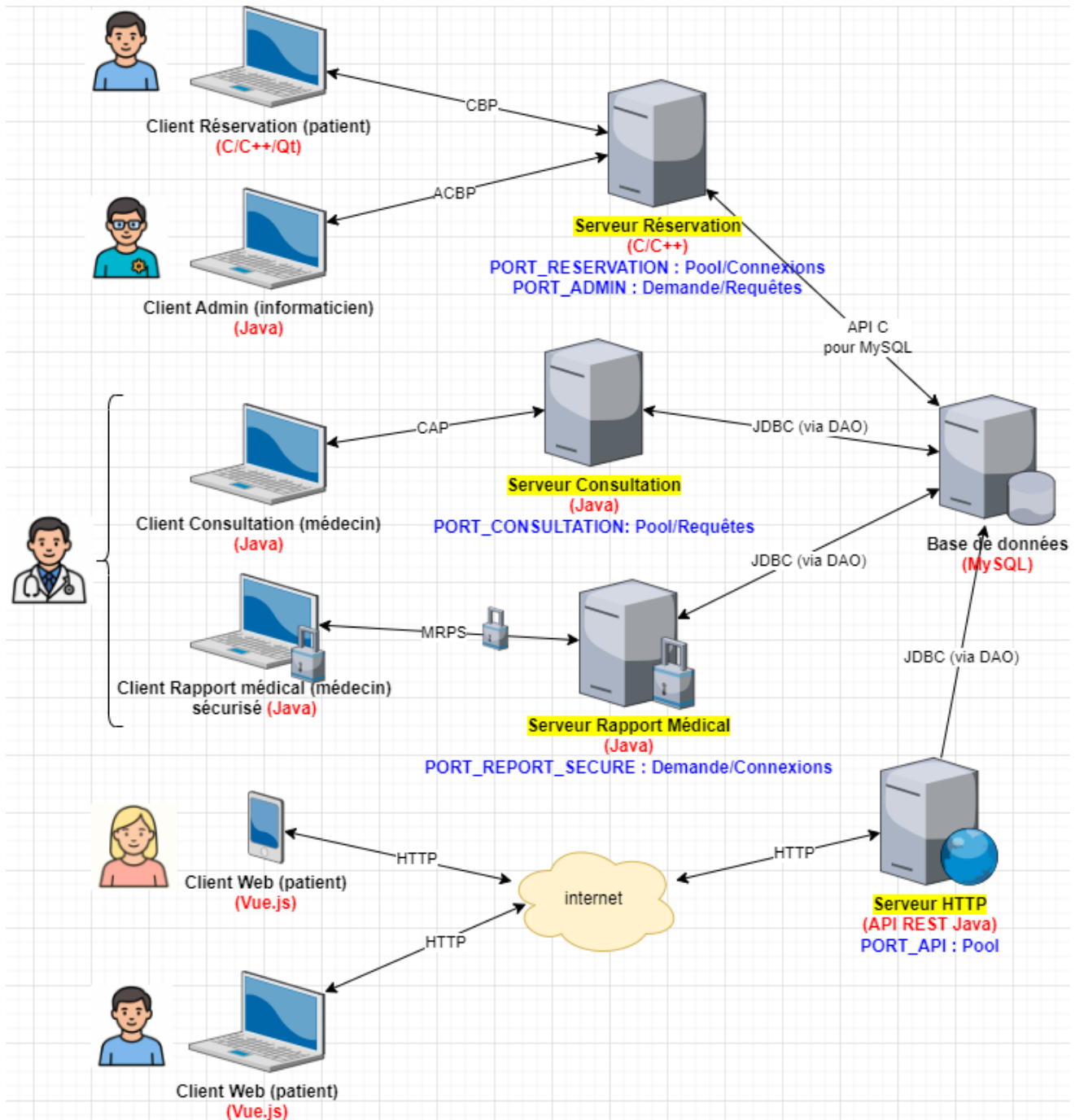
- Ce projet doit être réalisé par une équipe de 2 étudiants.
- Il y aura 3 évaluations de laboratoire :
 - 2 évaluations (continues) orales pendant le Q1 comptant chacune pour 25% de la cote de laboratoire, et
 - 1 évaluation orale pendant la session de janvier comptant pour 50% de la cote de laboratoire.
- Toutes les démonstrations de vos clients / serveurs devront être réalisées en réseau (machines physiques différentes, VM vers VM, ...)
- Construction de la cote globale de l'AA : moyenne géométrique entre la cote de l'examen de théorie de janvier (50%) et de la cote de laboratoire (50%)

Planning des évaluations :

Laboratoire	Contenus	Date
Evaluation 1 (continue, 25%)	Librairie de sockets C/C++, serveur multi-threads C/C++, client C (C++/Qt), base de données MySql	13/10/2025-17/10/2025
Evaluation 2 (continue, 25%)	Client Java pour le serveur C, DAO d'accès à la BD, serveur et client « Consultation » Java	10/11/2025-14/11/2025
Evaluation 3 (examen, 50%)	Serveur et client « Rapport Médical » Java sécurisé (crypto), API Java web, frontend en Vue.js 3	Date de l'examen de laboratoire de janvier 2026

Schéma global de l'application

L'application globale comporte plusieurs serveurs et clients écrits dans différents langages de programmation et paradigmes imposés. Voici le schéma global de l'application à réaliser :



Le développement de ce projet sera découpé en 3 parties (correspondant aux 3 évaluations) décrites ci-dessous.

Consigne importante concernant ChatGPT :

« **ChatGPT** (ou toute aide « IA » à la production automatique de code) est ton ami mais... Pour qu'il le reste, tu devras être capable d'expliquer tout le code généré par **ChatGPT**. Tout code généré par **ChatGPT** que tu seras incapable d'expliquer en détails débouchera sur une cote nulle concernant la question sur ce code (ET pour la partie concernée du projet), même si celui-ci est correct et fonctionnel. »

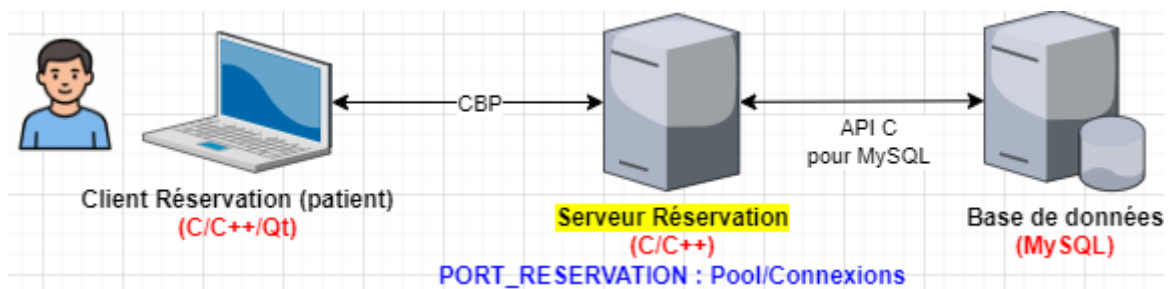
Partie / Evaluation 1

Date de remise : Semaine du 13/10/2025 au 17/10/2025

Cette partie comporte les éléments suivants :

- la construction d'une librairie de sockets en C/C++
- le serveur « Réservation » multi-threads C/C++ tournant sur une machine Linux (et utilisant la librairie de sockets développée)
- le client C/C++/Qt capable de communiquer avec ce serveur
- la mise en place de la base de données MySQL

Sur le schéma global de l'application, cela correspond à :



L'application « Client Réservation »

C'est grâce à l'application « Client Réservation » (tournant sur une des **machines Linux** présentes sur les bornes d'accueil de l'hôpital) qu'un patient pourra réserver une consultation avec un médecin. Tout ceci se fera bien sûr via un dialogue avec le « Serveur Réservation ».

Pour faciliter le développement de l'application client, l'interface graphique Qt vous sera fournie (lien GitHub : https://github.com/hepl-dacsc/LaboDACSC2025_sources)

La capture d'écran montre l'interface graphique de l'application « Réservation d'une consultation ». Elle est divisée en plusieurs sections :

- Coordonnées du patient :** Champs pour le nom (Wagner), le prénom (Jean-Marc) et le numéro de patient (8). Une case à cocher « Nouveau Patient » est présente.
- Recherche de consultations :** Champs pour la spécialité (Dermatologie), le médecin (Maboul Paul) et les dates (entre 15/09/25 et 31/12/25).
- Actions :** Boutons « Login », « Logout » et « Lancer la recherche ».
- Consultations trouvées :** Un tableau à 5 colonnes (Id, Spécialité, Médecin, Date, Heure) affichant 3 résultats.
- Footer :** Un grand bouton « Réserver ».

Id	Spécialité	Médecin	Date	Heure
1	Neurologie	Martin Claire	2025-10-01	09:00
2	Cardiologie	Lemoine Bernard	2025-10-06	10:15
3	Dermatologie	Maboul Paul	2025-10-23	14:30

Quelques explications :

- Avant toute chose, le patient doit s'identifier à l'aide de son nom, prénom et de son numéro de patient (pour simplifier les choses, il s'agira simplement de son identifiant en base de données). Deux cas de figure peuvent se présenter au moment du clic sur le bouton « **Login** » :
 - Le patient n'existe pas encore en BD. Il devra alors cocher « Nouveau Patient ». Le serveur lui attribuera donc un numéro de patient (un id donc) qui lui servira de « mot de passe »
 - Le patient existe déjà en BD. Il ne devra pas cocher « Nouveau Patient ». Il s'identifiera donc avec son nom, prénom et numéro de client (son id donc).
- Une fois entré « en session », le patient aura accès aux boutons « **Logout** », « **Lancer la recherche** » et « **Réserver** »
- Un clic sur le bouton « **Lancer la recherche** » récupérera la spécialité, le médecin et la fourchette de dates correspondant aux critères de recherches du patient. Le résultat de la recherche (les consultations disponibles, c'est-à-dire pas déjà réservées par d'autres patients) apparaîtra dans la table du dessous.
- Un clic sur le bouton « **Réserver** » récupérera la consultation sélectionnée dans la table. Après demande de la raison de la consultation (via une boîte de dialogue), la consultation lui sera attribuée et la raison stockée en BD.

L'application « Client Réservation » permet donc de modifier une consultation présente en BD (en lui affectant un patient et une raison) mais pas de créer une nouvelle consultation. Elle ne permet pas non plus d'annuler une réservation.

La base de données

Il s'agira d'une **base de données MySQL**. Vous pouvez la créer vous-même ou alors vous pouvez utiliser le programme **CreationBD** fourni. Ce programme (à utiliser sur la VM Oracle Linux fournie) créera les tables patients, specialties, doctors et consultations et les pré-remplira. Les autres tables sont à votre charge. Voici un bref descriptif de ces tables :

Nom Table	Champs
consultations	id (clé primaire), doctor_id (clé étrangère ne pouvant pas être NULL), patient_id (clé étrangère pouvant être NULL), date, hour, reason
doctors	id (clé primaire), specialty_id (clé étrangère ne pouvant pas être NULL), last name, first name
specialties	id (clé primaire), name
patients	id , last name, first name, birth date
reports	id , description, ... → à vous de voir
...	...

La manipulation de la base de données se fera en utilisant l'**API C/MySQL** utilisée en BAC 2 et décrite dans le syllabus « Système d'exploitation Linux – Programmation avancée en C », annexe 2.

La librairie de sockets

Dans un premier temps, on vous demande de développer une petite **librairie C ou C++** de sockets (fournie sous forme de fichiers .cpp et .h) dont les caractéristiques sont :

- elle doit être **générique** : on ne doit pas voir apparaître la notion de « patients » ou de « consultations » dans le prototype des fonctions. Elle doit pouvoir être réutilisée telle quelle dans une autre application

- elle doit être **abstraite** : l'utilisation de votre librairie doit permettre d'éviter de voir apparaître les structures systèmes du genre « sockaddr_in » dans les programmes qui utilisent votre librairie

Conseils / propositions pour le développement :

- Une proposition de prototypes de fonctions pour votre librairie est fournie dans un des pdf de théorie (« Communications Réseaux en C »)
- Vous avez le droit de développer cette librairie en C++. Néanmoins, on ne vous l'impose/le conseille pas car le développement d'une librairie correcte en C++ vous prendra plus de temps qu'en C
- Des tests élémentaires de votre librairie sont plus que bienvenus avant de l'embarquer dans votre client Qt ou votre serveur Réservation.

Le serveur Réservation

Le « Serveur Réservation » devra être un serveur

- **de connexions** (et non de requêtes)
- **multi-threads écrit en C (threads POSIX)**,
- implémentant le modèle **« pool de threads »**
- attendant sur le port PORT_RESERVATION
- tournant sur une **machine Linux** (pouvant être la VM Oracle Linux dont vous disposez déjà)

Le nombre de threads du pool ainsi que le PORT_RESERVATION devront être lus dans un **fichier (texte) de configuration** du serveur.

Vous devez savoir que, dans la seconde partie, un **client Java** de type **« administrateur »** se connectera sur un autre port du serveur Réservation afin de récupérer la liste des patients (ainsi que leur adresse IP) connectés sur les différentes bornes de réservation de l'hôpital. Il serait donc judicieux de mettre en place directement un système de mémorisation de ces informations.

Le protocole de communication entre le « Client Réservation » et le « Serveur Réservation » s'appellera **« Consultation Booking Protocol » (CBP)**. Les différentes commandes sont décrites ci-dessous :

Serveur Réservation -- Protocole CBP – PORT_RESERVATION			
Commande	Requête	Réponse	Actions / Explications
« LOGIN »	Nom, prénom, no patient (ou pas), nouveau patient	Oui ou non (accompagné du numéro de patient si nouveau)	Vérification de l'existence du patient sur base du nom, prénom et no patient / création d'un nouveau patient en BD
« LOGOUT »	...	...	...
« GET_SPECIALTIES »		Liste des spécialités (id, name)	Récupération des spécialités médicales en BD
« GET_DOCTORS »		Liste des médecins (id, lastName, first_name)	Récupération des médecins en BD

« SEARCH_CONSULTATIONS »	Spécialité, médecin, date début, date de fin	Liste des consultations (id, médecin, spécialité, date, heure)	Récupération des consultations disponibles en BD et répondant aux critères reçus
« BOOK_CONSULTATION »	consultationId, reason	Oui ou non	Mise à jour BD de la consultation choisie (id du patient + raison). Si plus dispo, retour « non »

Partie / Evaluation 2

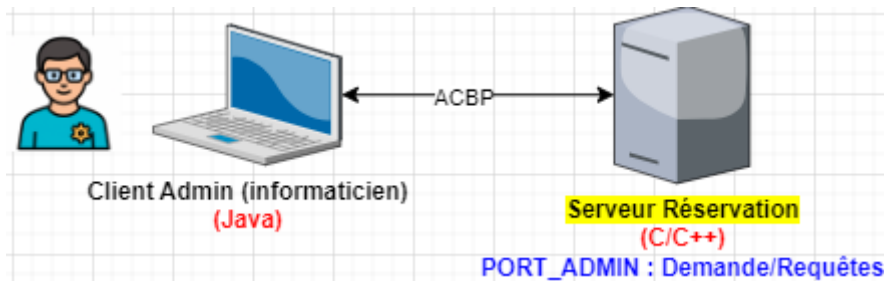
Date de remise : Semaine du 10/11/2025 au 14/11/2025

Cette partie comporte les éléments suivants :

- le client Java « Admin » pour le serveur Réservation
- les DAO d'accès à la base de données construits en utilisant JDBC
- le serveur « Consultation » multi-threads Java, utilisant les DAO développés
- le client Java « Consultation » capable de communiquer avec ce serveur

L'application « Client Admin » Java

Sur le schéma global de l'application, cela correspond à :



Vous devez développer ici une application fenêtrée en **Java (Swing)** capable d'interagir avec le « Serveur Réservation » déjà développé. Cette application cliente, utilisée par les administrateurs en informatique de l'hôpital, se connectera sur le port **PORT_ADMIN** du serveur Réservation. Le protocole, « **Admin Consultation Booking Protocol** » (ACBP) est des plus simples car ne comportant qu'une seule commande :

Serveur Réservation -- Protocole ACBP – PORT_ADMIN

Commande	Requête	Réponse	Actions / Explications
« LIST_CLIENTS »		Liste des clients connectés (adresse IP, nom, prénom, no patient)	Interrogation du protocole CBP afin d'obtenir cette liste

Le serveur traitant le protocole ACBP sera un serveur

- **de requêtes** (et non de connexions)
- **multi-threads écrit en C (threads POSIX)**,
- implémentant le modèle « **à la demande** »
- attendant sur le port PORT_ADMIN

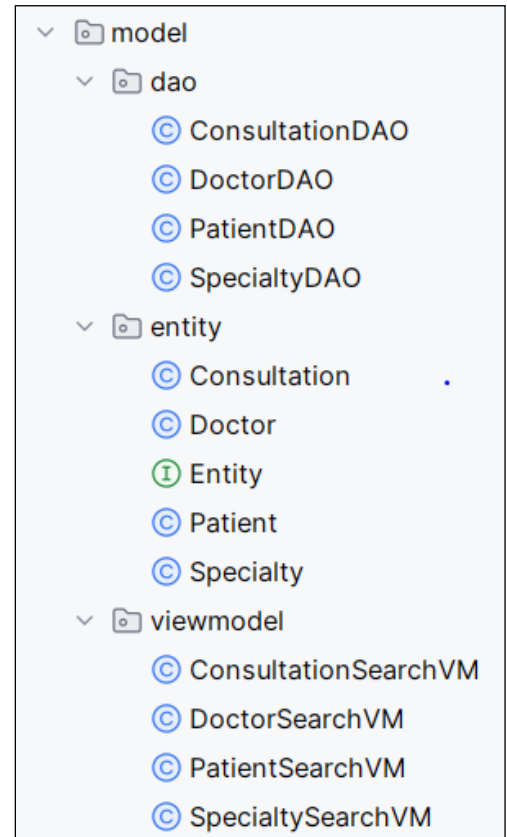
- tournant au sein du **même processus** que celui s'occupant du protocole CBP → le processus « Serveur Réservation » comportera donc deux sockets d'écoute (une sur le PORT_RESERVATION et une sur le PORT_ADMIN).

Les DAO (Java) d'accès à la base de données

L'accès à la base de données ne devra pas se faire avec les primitives **JDBC** utilisées telles quelles, mais plutôt au moyen d'objets permettant d'accéder aux données, des **DAO** (« **Data Access Object** ») spécifiques à chaque entité.

On demande donc de construire un ensemble de classes et de packages :

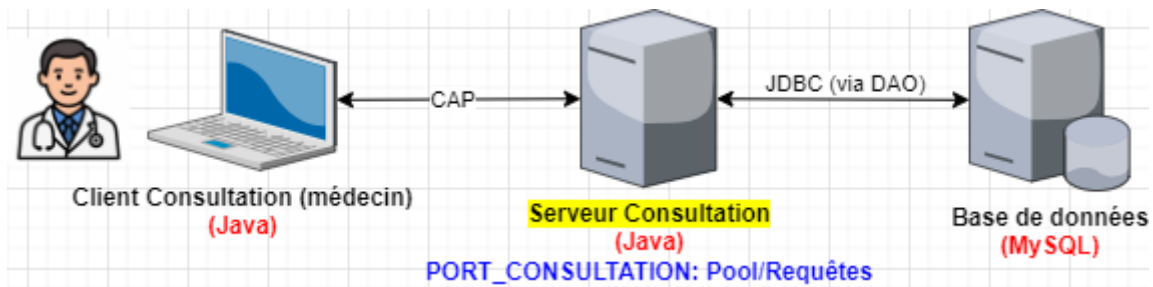
- les classes « **entity** », c'est-à-dire les objets « métiers » : **Patient**, **Doctor**, **Consultation**, ... collant (« **object mapping** ») à la structure de la base de données
- Les classes « **DAO** » encapsulant la connexion à la base de données et les méthodes classiques dites « **CRUD** »
- Les classes « **xxxSearchVM** » permettant de réaliser des sélections d'entités selon certains critères. En particulier, on pourra réaliser des filtres de recherche sur
 - les « **consultations** » selon l'id, le médecin et le patient.
 - les « **patient** » selon l'id, le lastName (« like ») et le firstName (« like »)
 - les « **médecins** » selon l'id, la spécialité (« like »), le lastName (« like »)
 - ... selon les besoins que vous aurez pour répondre aux fonctionnalités demandées



Attention qu'une seule instance de la connexion à la base de données devra être utilisée par tous les objets DAO instanciés (la connexion à la BD sera donc unique). Chaque type d'objet **xxxDAO** ne sera instancié qu'une seule fois et partagé entre les threads du serveur Consultation. Ce serveur est un serveur multi-threads : attention donc aux accès concurrents !

Le serveur Consultation et son application cliente

Sur le schéma global de l'application, cela correspond à :



L'application Java « Client Consultation » est utilisée par un médecin qui souhaite gérer ses consultations. En voici les fonctionnalités :

- Pour entrer en session, un médecin devra encoder un login et un mot de passe → vous êtes libres de modifier la structure de la base de données ou d'ajouter des tables
- Le médecin pourra alors :
 - Lister l'ensemble de SES consultations (pas celles des autres médecins) qu'elles soient déjà réservées par un patient ou non (dans une **JTable** où chaque ligne correspondra à une consultation → on devra y voir apparaître l'id, le patient (ou pas), la raison (ou pas), la date et l'heure) → il pourra également filtrer ses consultations sur les critères suivants : **patients** et **dates**
 - Encoder un nouveau patient (nom et prénom) → celui-ci sera ajouté en BD et un id lui sera attribué
 - Créer une ou plusieurs nouvelles consultations (pour lui, pas pour un autre médecin) en choisissant la date et l'heure de début → la (les) consultation(s) sera(ont) ajoutée(s) en BD → il pourra ensuite lui-même encoder le patient et la raison ou alors laisser la consultation libre d'être réservée par un patient
 - Supprimer une de ses consultations qui n'a pas encore été réservé par un patient.
 - Modifier une de ses consultations (date, heure mais pas le patient ni la raison).

A vous à créer une application graphique (**Swing**) qui soit le plus ergonomique possible pour répondre à ces fonctionnalités.

Etant donné, que l'application Java « Client Consultation » et le « serveur Consultation » se situent tous les deux dans le même réseau local et qu'aucune donnée critique des patients ne transite entre eux, aucun mécanisme de sécurisation (cryptage, signature électronique, ...) n'a été envisagé ici.

Consignes d'implémentation

Tout d'abord, le pont JDBC entre le serveur Consultation et la base de données se fera en utilisant les DAO développés à la section précédente.

Le « Serveur Consultation » devra être un serveur

- **multi-threads écrit en Java (utilisant le package java.net),**
- **de requêtes** (et non de connexions)
- implémentant le modèle « **en pool de threads** »
- attendant sur le port **PORT_CONSULTATION**

Le numéro du port PORT_CONSULTATION, les paramètres de la BD seront lus dans un **fichier (properties) de configuration** du serveur.

Pour la construction du protocole, vous utiliserez des **objets sérialisés** vu que c'est de la **communication Java-Java**. Ce protocole s'appellera « **Consultation Administration Protocol** » (CAP). Les différentes commandes sont décrites ci-dessous :

Serveur Consultations -- Protocole CAP – PORT CONSULTATION			
Commande	Requête	Réponse	Actions / Explications
« LOGIN »	Login et password du médecin	Oui ou non	Vérification du login/password du médecin en BD (...)
« ADD_CONSULTATION »	Date, heure, durée, nombre de consultations consécutives	Oui ou non (si on dépasse 17h00)	Création en BD du nombre de consultations souhaitées pour ce médecin, à la date fournie et à partir de l'heure fournie (la durée permet de connaître les heures de chaque entité consultation créée en BD) → actuellement aucun patient ni raison attribuée
« ADD_PATIENT »	Nom et prénom du patient	Id attribué au patient	Ajout du patient en BD
« UPDATE_CONSULTATION »	Id de la consultation, nouvelle date, nouvelle heure, patient, raison	Oui ou non	Attribue un patient et une raison à une consultation existante OU modifie les date/heure d'une consultation déjà attribuée à un patient
« SEARCH_CONSULTATIONS »	Patient, date	Liste des consultations	Recherche en BD des consultations (de ce médecin) selon les critères demandés (patient, date)
« DELETE_CONSULTATION »	Id de la consultation	Oui ou non	Suppression de la consultation si pas déjà attribuée à un patient
« LOGOUT »	...	...	...

Partie / Evaluation 3

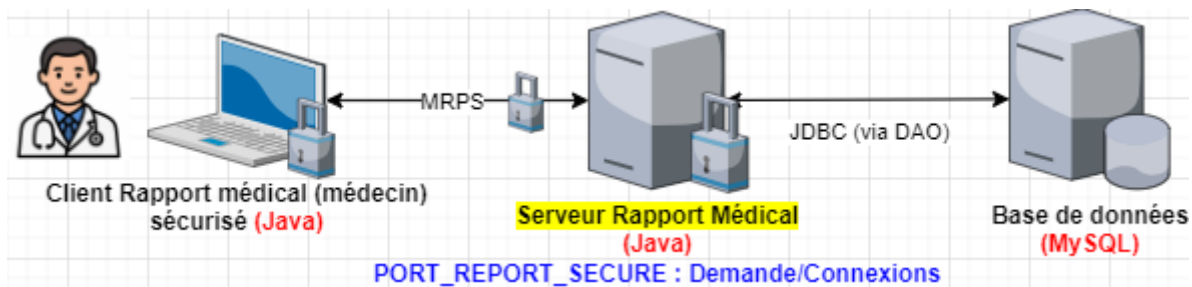
Date de remise : jour de l'examen de laboratoire en janvier 2026

Cette partie comporte les éléments suivants :

- le « Serveur Rapport Médical » sécurisé utilisant les outils cryptographiques, et l'application « Client Rapport Médical » correspondante
- une API REST Web (Java) permettant des opérations sur les patients, les médecins, les spécialités et les consultations
- le frontend en Vue.js consommant l'API créée

Le « Serveur Rapport Médical » et son application cliente

Sur le schéma global de l'application, cela correspond à :



L'application « **Client Rapport Médical** » sera utilisée par les médecins qui, au terme d'une consultation, doivent encoder un rapport médical concernant cette consultation. En voici les fonctionnalités :

- Pour entrer en session, un médecin devra encoder son login et son mot de passe
- Le médecin pourra alors :
 - Encoder un rapport (un simple texte) qui sera associé en BD au patient traité → vous devez donc ajouter une table « **reports** » à la BD → cette table comportera une clé primaire pour l'**id**, une clé étrangère pour l'**id du médecin**, une clé étrangère pour l'**id du patient**, une date (celle du rapport qui peut aussi être celle de la consultation) et le texte du rapport
 - Modifier un de ses rapports médicaux
 - Lister l'ensemble de tous SES rapports (dans une **JTable** où chaque ligne correspond à un rapport → il devra être possible de consulter le rapport dans une **JTextArea** car ce texte peut ne pas tenir dans une case de la **JTable**)
 - Lister l'ensemble des rapports concernant un de ses patients dont il connaît le numéro de patient (son id donc)

Cette application cliente va dialoguer avec le « Serveur Rapport Médical » mais sur le port dédié aux communications sécurisées (**PORT_REPORT_SECURE**). Le protocole qui va être mis en place va mettre en œuvre les outils cryptographiques classiques : cryptages symétrique et asymétrique, signature électronique, HMAC, ...

Le « Serveur Achat Rapport Médical » » devra être un serveur

- **multi-threads écrit en Java**,
- **de connexions** (et non de requêtes)
- implémentant le modèle « **à la demande** »
- utilisant les outils cryptographiques de la librairie Java **Bouncy Castle**
- attendant sur le port **PORT_REPORT_SECURE**

Le nombre de threads du pool ainsi que le **PORT_REPORT_SECURE** seront lus dans un **fichier (properties) de configuration** du serveur.

De nouveau, l'application Java cliente sera développée en **Swing**, être le plus ergonomique possible et permettra au médecin de communiquer avec le serveur Rapport Médical selon le protocole décrit ci-dessous.

Le protocole utilisé s'appellera « **Medical Reports Protocol Secure** » (MRPS). Les différentes commandes sont décrites ci-dessous :

Serveur Rapport Médical sécurisé -- Protocole MRPS – PORT REPORT SECURE			
Commande	Requête	Réponse	Actions / Explications
« LOGIN »	Login du médecin + ... avec digest salé (voir ci-dessous) → handshake pour l'envoi d'une clé de session	Oui ou non → réception d'une clé de session	Le password du médecin ne peut pas transiter en clair sur le réseau → digest salé Vérification du login/password en BD (...)
« ADD_REPORT »	Date, id du patient, rapport cryptés symétriquement + signature du médecin	Oui ou non	Ajout du rapport en BD à condition que ce patient ait au moins une consultation avec le médecin
« EDIT_REPORT »	Id du rapport et rapport modifié cryptés symétriquement	Oui ou non	Modification du rapport correspondant
« LIST_REPORTS »	Id du patient (ou pas), ...	Liste des rapports cryptés symétriquement + HMAC de la réponse	Recherche en BD des rapports correspondants
« LOGOUT »	...	...	...

Pour la commande LOGIN :

1. le médecin envoie son login,
2. le serveur vérifie l'existence du médecin via le login reçu. Si celui-ci existe, le serveur génère le « sel » et l'envoie à l'application cliente,
3. l'application cliente génère un **digest** à partir du login, du password encodé par médecin et du sel reçu du serveur, digest qu'elle envoie au serveur,
4. le serveur vérifie le digest reçu sur base du sel, du login et du password présent en BD.

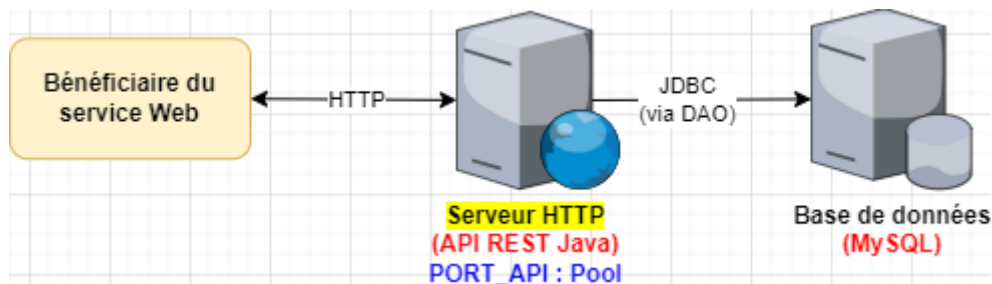
L'échange (**handshake**) d'une clé de session (**cryptée asymétriquement**) et la signature électronique nécessite un couple de clés privée/publique. Dans un premier temps, vous pourrez vous contenter de fichiers sérialisés (au préalable créés dans une application dédiée) et connus du client et du serveur. Néanmoins, dans un second temps, plusieurs solutions peuvent être envisagées :

- le client et le serveur disposent chacun d'un keystore → Le client possède dans son keystore son couple de clés privée/publique tandis que le serveur dispose dans son keystore du certificat du client. A des fins de simplification du développement, on pourra se contenter d'un seul couple de clés privée/publique pour toutes les instances de l'application client
- lors de la phase de login, le client se génère un couple de clés privée/publique et envoie au serveur sa clé publique (mieux : son certificat)

L'API REST de gestion des spécialités, des médecins, des patients et des consultations

Il s'agit ici de développer un service Web de type API REST qui permettra à toute personne (ou « bénéficiaire du service Web ») disposant de l'URL de réaliser des opérations (CRUD (et autres) sur les patients, les spécialités, les médecins et les consultations présents dans la base de données.

Sur le schéma global de l'application, cela correspond à :



Le protocole de l'API REST à développer est le suivant :

Routes / URL	Requête	Réponse
/api/specialties	Méthode : GET Query : / Body : /	Liste de toutes les spécialités
/api/doctors /api/doctors?name=Maboul /api/doctors?specialty=Neurologie	Méthode : GET Query : critères de recherche Body : /	Liste de tous les médecins trouvés selon les critères
/api/patients	Méthode : POST Query : / Body : lastName, firstName, birthDate, patientId, newPatient	Vérifie existence ou ajoute nouveau patient en fonction de newPatient (true ou false) et retourne une réponse positive (accompagnée de l'id du patient) ou négative
/api/consultations /api/consultations?date=2025-10-15 /api/consultations?doctor=Maboul /api/consultations?specialty=Neurologie /api/consultations?patientId=249	Méthode : GET Query : critères de recherche Body : /	Liste des consultations <u>non encore réservées</u> et répondant aux critères. Si <u>patientId</u> est précisé, on retourne alors les consultations réservées par le patient

/api/consultations?id=3	Méthode : PUT Query : id de la consultation à modifier Body : no de patient, raisons	Modifie la consultation d'id fourni en BD en lui attribuant un patient et une raison et retourne une réponse positive ou négative
/api/consultations?id=3	Méthode : DELETE Query : id de la consultation à supprimer Body = /	Modifie la consultation d'id fourni → libère le champ patientId et la raison en BD → la consultation redevient disponible pour un autre patient. Retourne oui ou non

Remarquez que

- le body des requêtes devra être soit au **format url-encoded** (application/x-www-form-urlencoded) ou au **format JSON** (application/json)
- le body des réponses devra être au **format JSON** (application/json)
- lors d'une sélection (méthode **GET**), les critères de recherche contenus dans la query ne doivent pas nécessairement comporter tous les champs mais un sous-ensemble de ceux pour lesquels les xxxSearchVM ont été configurés

Consignes d'implémentation

L'**API REST** devra être développée en **Java « from scratch »**, c'est-à-dire sans avoir recours à un serveur d'applications comme Apache Tomcat ou autre. On vous demande d'utiliser la classe **HttpServer** et l'interface **HttpHandler** du **package Java com.sun.net.httpserver**.

Votre API REST ainsi créée accèdera à la base de données via les **DAO** déjà développés.

Les tests de votre API seront réalisés avec le logiciel **Postman** (<https://www.postman.com/>) et/ou la commande **http** de la machine Oracle Linux fournie. Exemple :

```
# http GET 'http://192.168.0.23:8080/api/specialties'
...
```

Le frontend de l'application Web

L'application Web à développer ici sera utilisée par un patient souhaitant

- prendre rendez-vous pour une consultation avec un médecin
- consulter les rendez-vous de consultation qu'il a déjà pris en ligne ou à l'hôpital à l'aide des bornes d'accueil
- annuler un rendez-vous

Cette application sera développée avec **Vue.js 3** et sera une application du type « **SPA** » (Single Page Application).

L'unique page devra comporter **plusieurs composants Vue** :

1. un composant permettant à l'utilisateur d'encoder
 - son nom, prénom et numéro de patient s'il est déjà encodé dans la base de données de l'hôpital
 - son nom, son prénom et le flag « nouveau patient » (→ via une checkbox) s'il n'est pas encore connu de l'hôpital

Ce composant est donc fort similaire à la zone en haut à gauche du client Qt fourni et permet au patient d'« **entrer en session** »

Au tout début, ce composant sera le seul à apparaître sur la page web. Un clic sur le bouton « **Login** » enverra une requête à l'API afin que le patient puisse s'identifier. Une fois identifié, le second composant ci-dessous doit apparaître (en plus du premier composant)

2. un composant affichant la **liste des rendez-vous de consultations déjà réservés pour le patient connecté**. Cette liste apparaîtra sous forme d'une table contenant pour chaque consultation :
 - la date du rendez-vous
 - l'heure du rendez-vous
 - le médecin
 - la spécialité
 - la raison du rendez-vous

En-dessous de cette table devront se trouver plusieurs boutons :

- « **Logout** » : pour sortir de session et faire disparaître ce composant et remettre la page dans son état initial
- « **Supprimer** » : pour supprimer (après confirmation via une boîte de dialogue) la consultation sélectionnée dans la table → annule le rendez-vous, c'est-à-dire libère la consultation qui sera alors disponible pour un autre patient
- « **Prendre un autre rendez-vous** » : un clic sur ce bouton fera apparaître le 3^{ème} composant décrit ci-dessous

3. Un composant affichant, sous forme d'une table, **toutes les consultations disponibles (non encore réservées par un patient)**. On devra également voir apparaître
 - Un **combobox** contenant toutes les spécialités (récupérées via l'API)
 - Un **combobox** contenant tous les médecins (récupérées via l'API)

L'utilisateur pourra faire une **recherche de sélection** sur la spécialité et/ou le médecin.

En-dessous de cette table devront se trouver plusieurs boutons :

- « **Réserver** » : pour réserver la consultation sélectionnée dans la table. Après avoir encodé la raison (via boîte de dialogue ou autre), le composant devra disparaître et la consultation

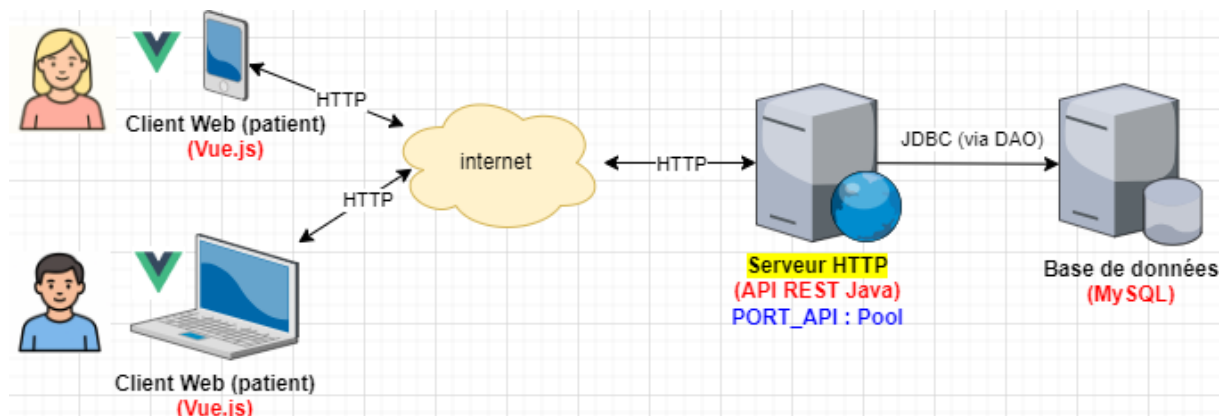
fraichement réservée devra apparaître dans le 2^{ème} composant (liste des consultations réservées par le patient)

- « **Annuler** » : pour faire disparaître ce composant → reste alors sur la page le premier et le second composant.

Il est clair que l'application web devra fonctionner de pair avec l'API REST et la plupart des clics sur les boutons de l'interface graphique enverra une requête HTTP vers l'API.

Consignes d'implémentation

Sur le schéma global de l'application, cela correspond à :



Au niveau **backend**, il s'agit donc de **l'API REST Web Java** décrite ci-dessus. Vous pouvez la compléter si le besoin s'en fait sentir.

Au niveau **frontend**, il s'agit de **l'application Vue.js 3** décrite ci-dessus. Celle-ci devra respecter les contraintes techniques suivantes :

- Une seule page HTML, index.html, mais comportant plusieurs composants Vue.js communiquant entre eux via App.vue, les « **props** » et les **événements**.
- L'utilisation de **TypeScript** dans tout le projet
- L'utilisation de l'**API Composition de Vue.js 3**
- Le respect des principes SOLID en implémentant plusieurs **DAO** pour les différentes entités (spécialités, médecins, patients et consultations). Ces DAO contiendront les envois et réceptions des requêtes HTTP vers l'API.
- Les appels vers l'API se feront à l'aide de la fonction **fetch()** de Javascript.
- On vous demande également de gérer le style de votre application pour qu'elle soit le plus ergonomique possible, soit via **votre propre code CSS**, soit via **Bootstrap** ou toute autre librairie de style (en accord avec professeurs de laboratoire).

Aller plus loin ?

Voici quelques bonus disponibles :

BONUS 1 : Réalisez une seconde version de **l'API REST Java** en utilisant **Spring Boot** → à vous à vous documenter sur le sujet

BONUS 2 : Modifier votre **application Vue.js** de telle sorte qu'elle comporte plusieurs « vues », c'est-à-dire plusieurs routes → utilisation de **Vue Router** mais également de **Pinia** ou **Vuex** pour le partage de données entre les vues → à vous à vous documenter sur le sujet

N'hésitez pas à demander conseil à vos professeurs de laboratoire pour tout choix d'implémentation.

Bon travail 😊 !